Accelerating Magnetic Particle Imaging with Data Parallelism: A Comparative Study

Haipeng Zhu[1], Yuan He[2*], Xiaohan Yue[1], Qinyang Zhang[1], Lihui Wang[3] Shi Bai[1,4]
haipengzhu@smail.sut.edu.cn, isaacyhe@acm.org, xhyue@sut.edu.cn
qyzhang@smail.sut.edu.cn, wanglihui@suda.edu.cn, baishi@sut.edu.cn
[1]Shenyang University of Technology, Shenyang, Liaoning, China
[2]Keio University, Yokohama, Kanagawa, Japan
[3]Soochow University, Suzhou, Jiangsu, China
[4]Liaoning Jiayin Medical Technology Co., Ltd., Shenyang, Liaoning, China

***Abstract*—Magnetic Particle Imaging (MPI) is an emerging technology with significant potential for real-time clinical applications. However, computational demands of non-negative least squares (NNLS) reconstruction present a critical bottleneck limiting real-time deployment. We address these challenges by implementing and comparing multiple acceleration strategies for NNLS in MPI. In this paper, two major NNLS algorithms, active set and gradient projection methods, are investigated for cases including the baseline MATLAB implementation, multi-core CPU-based accelerations with OpenMP, GPU accelerations using CUDA, and GPU implementations with unified memory for larger datasets. Performance evaluations are carried out with both single (FP32) and double precision floating point (FP64) data across problem sizes from 21×21 to 301×301 representing typical system matrices in MPI. Results demonstrate substantial performance gains across all acceleration strategies. GPU implementations achieve remarkable improvements over multi-threaded MATLAB baseline, resulting in more than two orders of magnitude acceleration under FP32 for system matrix sizes of up to 131×131; while multi-core CPU implementations with OpenMP deliver up to 31× speedup under FP64 for 301×301 system matrix. For large datasets exceeding GPU memory (VRAM), unified memory enables GPU accelerations of up to 34-fold under FP32, making previously intractable problems feasible while maintaining scalability.**

***Index Terms*—Magnetic particle imaging, 3D reconstruction, non-negative least squares, hardware acceleration, data parallelism**

I. Introduction

Magnetic Particle Imaging is an emerging medical modality [1] offering real-time, high-resolution visualization of superparamagnetic iron oxide nanoparticles (SPIONs) for a range of clinical and research applications [2], [3]. Unlike conventional imaging techniques such as CT or MRI, MPI directly detects tracer materials with zero tissue background signal, enabling high contrast imaging at any depth. This unique capability, combined with the absence of ionizing radiation and excellent temporal resolution, makes MPI promising for interventional procedures requiring real-time feedback.

MPI-guided sentinel lymph node biopsy (SLNB) has shown promise for improving surgical accuracy in breast and other cancers [4]. Additional emerging applications include cardiovascular imaging, stroke detection, and stem cell tracking, all demanding sub-second reconstruction times. However, the core challenge is the computational cost of image reconstruction, which relies on solving large-scale non-negative least squares (NNLS) problems [5].

The computational intensity stems from multiple factors: iterative algorithms requiring dense matrix-vector multiplications with millions of elements, dozens to hundreds of iterations for convergence, and non-negativity constraints necessitating projection operations at each iteration. For 3D imaging or multi-coil data, reconstructions routinely involve system matrices with hundreds of thousands to millions of unknowns. High-resolution 3D reconstructions with multi-channel reception generate system matrices exceeding several gigabytes, presenting both computational throughput and memory bandwidth challenges.

Despite advances in MPI scanner hardware [2], [6], real-time clinical deployment remains fundamentally limited by NNLS reconstruction performance. Conventional CPU-based implementations require seconds to minutes per scan [5], far exceeding the sub-second latency needed for intraoperative guidance. This bottleneck prevents MPI from realizing its full potential as an interventional imaging tool.

Previous work in hardware-accelerated medical imaging has demonstrated the potential of multi-core CPUs and GPUs for modalities such as CT [7], MRI [8], and PET [9], delivering one to two orders of magnitude speedup. However, systematic evaluations targeting MPI remain limited, particularly regarding NNLS-specific characteristics, memory scalability for large-scale 3D reconstructions, and numerical precision requirements. MPI's dense system matrices and strict non-negativity constraints differ from sparse iterative methods in CT or MRI, requiring different optimization strategies for parallel architectures.

This work addresses these challenges by implementing and systematically evaluating parallel NNLS reconstruction algorithms for MPI on modern hardware platforms. Specifically, we develop and compare:

- Multi-core CPU (OpenMP) and GPU (CUDA) acceleration for both active-set and gradient projection NNLS

* This author is currently with RIKEN Center for Computational Science, Tokyo, Japan.

algorithms;

- GPU unified memory strategies (managed, advice, prefetch, and combined) for out-of-VRAM reconstruction;
- Single- and double-precision computation to assess performance-accuracy trade-offs;
- Comprehensive benchmarking against MATLAB's `lsqnonneg` solver.

Our experiments cover clinically relevant system matrix sizes (up to 301×301 per slice) and include rigorous analysis of speedup and accuracy. Results demonstrate that GPU acceleration, especially in single precision, enables over two orders of magnitude improvement compared to MATLAB. Unified memory strategies successfully extend GPU applicability to data sizes exceeding VRAM, while OpenMP provides substantial acceleration on widely available platforms. By providing a comparative evaluation of algorithmic and architectural choices, this work establishes new baselines and practical guidance for deploying real-time MPI in research and clinical settings.

This paper is organized as follows: Section II provides technical backgrounds, Section III details our acceleration strategies, Section IV presents experimental evaluations, and Section V concludes this work.

II. Backgrounds

A. Magnetic Particle Imaging

MPI is a novel imaging modality that directly detects SPIONs with high sensitivity and zero background signal. MPI scanners employ selection, drive, and focus fields to define imaging geometry and spatial encoding [1]. The selection field gradient typically ranges from 2–12 T/m, directly determining the spatial resolution capabilities of the system. Drive field frequencies operate between 1–100 kHz, with higher frequencies enabling faster imaging but demanding more sophisticated hardware and increased computational throughput [6].

Two primary scanner architectures exist: field-free point (FFP) and field-free line (FFL) systems. FFP scanners provide true three-dimensional encoding through mechanical movement or electronic steering of the field-free region, generating dense spatial coverage at the cost of longer acquisition times [2]. FFL systems achieve significantly faster imaging speeds through rapid electronic scanning along one dimension but may sacrifice spatial resolution in the selection direction. Multi-channel receiver coils enable parallel data acquisition, improving signal-to-noise ratios through coil combination techniques while substantially increasing data volume [3].

SPION characteristics critically impact reconstruction complexity. Core sizes of 10–30 nm exhibit different relaxation behaviors (Néel and Brownian mechanisms) [10], determining optimal excitation frequencies and the size of the resulting system matrices [2]. The raw signals undergo filtering, amplification, and analog-to-digital conversion [3], producing datasets that scale quadratically with spatial resolution and linearly with the number of receiver coils [2], [5], creating substantial computational demands for real-time reconstruction.

B. Non-Negative Least Squares in MPI Reconstruction

The MPI reconstruction problem is formulated as a constrained optimization that accounts for the physical requirement that particle concentrations cannot be negative:

$$\mathbf{x}^* = \arg\min_{\mathbf{x}\geq 0} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 \tag{1}$$

where $\mathbf{A}$ represents the augmented system matrix and $\mathbf{b}$ is the augmented measurement vector. In practice, Tikhonov regularization is incorporated to stabilize the ill-posed inverse problem:

$$\mathbf{x}^* = \arg\min_{\mathbf{x}\geq 0} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 + \lambda\|\mathbf{x}\|_2^2 \tag{2}$$

Several algorithmic approaches exist for solving NNLS problems. Classical methods include the Lawson-Hanson active-set algorithm [11], which maintains and updates active constraints through QR factorizations, and coordinate descent methods [12]. Gradient-based approaches such as projected gradient methods [13] enforce constraints through projection $\mathcal{P}_{\mathbb{R}^n_+}(\mathbf{x}) = \max(0, \mathbf{x})$ onto the non-negative orthant. More recent developments include fast iterative shrinkage-thresholding algorithms [14] and alternating direction methods of multipliers [15]. Active-set methods like TNT-NN [16] provide efficient alternatives for large-scale problems.

In MPI applications, the choice of NNLS algorithm significantly impacts reconstruction speed and quality. Each iteration requires multiple matrix-vector multiplications with dense system matrices of size $M \times N$, where M represents measurements and N the reconstruction voxels. For clinically relevant problem sizes, system matrices contain hundreds of thousands of elements, requiring millions of floating-point operations per iteration. Convergence typically demands dozens to hundreds of iterations, leading to reconstruction times exceeding several seconds on conventional processors. Memory bandwidth compounds computational intensity as dense operations require streaming large data volumes.

Meanwhile, the dense matrix operations, element-wise constraint projections, and iterative structure present substantial opportunities for hardware acceleration through parallel processing. Unlike sparse methods in CT or MRI, MPI's dense system matrices offer abundant data-level parallelism without irregular memory access patterns.

C. Related Works

Hardware acceleration of medical imaging reconstruction has been extensively studied across various modalities and computational platforms.

1) GPU Acceleration in Medical Imaging: GPU acceleration has demonstrated substantial performance improvements across imaging modalities. In computed tomography, Xu *et al.* achieved 10-100× speedups for filtered backprojection and iterative reconstruction algorithms [7], while Tian *et al.* extended these improvements to statistical reconstruction methods [17]. Magnetic resonance imaging reconstruction benefits similarly, with Stone *et al.* demonstrating 10-50× speedups for compressed sensing algorithms on non-Cartesian k-space

data [8]. Positron emission tomography reconstruction has also leveraged GPU computing effectively, with implementations achieving real-time processing capabilities [9].

However, these prior efforts focus primarily on sparse matrix operations or Fourier-domain processing, whereas MPI reconstruction involves dense matrix operations with non-negativity constraints, presenting different optimization challenges.

2) Constrained Optimization Acceleration: Related constrained optimization problems have been successfully accelerated on parallel architectures. Parallel NNLS implementations have been developed for multi-core and GPU architectures, with Luo *et al.* demonstrating efficient GPU implementations of the active-set method using specialized QR updating strategies [18]. Non-negative matrix factorization implementations have achieved significant speedups for audio source separation [19], while hyperspectral imaging applications employ GPU-accelerated NNLS-based spectral unmixing algorithms [20]. These works emphasize the importance of memory access pattern optimization for peak GPU performance.

Despite these efforts, systematic evaluation of multiple NNLS algorithms across different precision levels specifically for MPI reconstruction remains absent from the literature.

3) Alternative Acceleration Platforms: Field-Programmable Gate Arrays offer potential advantages in power efficiency and latency for iterative reconstruction algorithms, though implementation complexity remains challenging compared to GPU approaches [21].

4) MPI-Specific Optimizations: Current MPI research primarily focuses on algorithmic improvements including system matrix-based and x-space methods, reconstruction quality enhancements, and deep learning approaches [5]. Hardware development has concentrated on expanding clinical capabilities, such as human-size MPI systems for applications like sentinel lymph node biopsy [4]. Despite these advances, most reconstruction algorithms require seconds to minutes per scan, creating a critical bottleneck for real-time clinical deployment.

A few preliminary studies have explored GPU acceleration for MPI, but these have been limited in scope without systematic evaluation of algorithm choices, numerical precision trade-offs, or memory scalability strategies. This computational challenge motivates our work, which addresses MPI-specific acceleration through systematic evaluation of parallel architectures across algorithms, platforms, and precision levels.

III. Accelerating MPI with Data Parallelism

A. MPI Reconstruction Workflow and Motivation

The MPI reconstruction pipeline is shown in Fig. 1. The process begins with calibration data of the point spread function (PSF) and raw MPI signals, processed into the system matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and measurement vector $\mathbf{b} \in \mathbb{R}^m$. The system matrix encodes the relationship between spatial particle distribution and frequency-domain measurements, constructed through calibration scans or model-based approaches. The measurement vector contains signal harmonics after filtering and frequency selection.

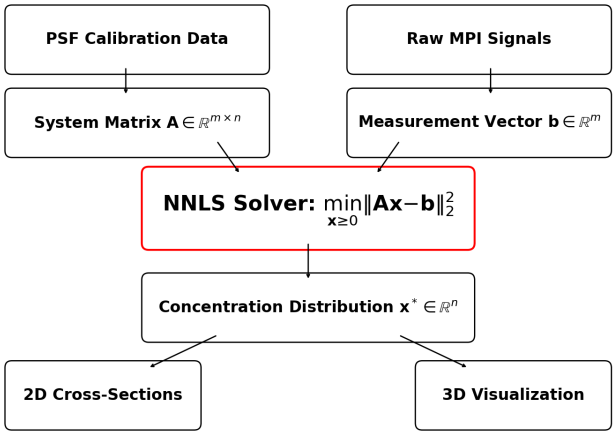


Fig. 1: Workflow of MPI reconstruction.

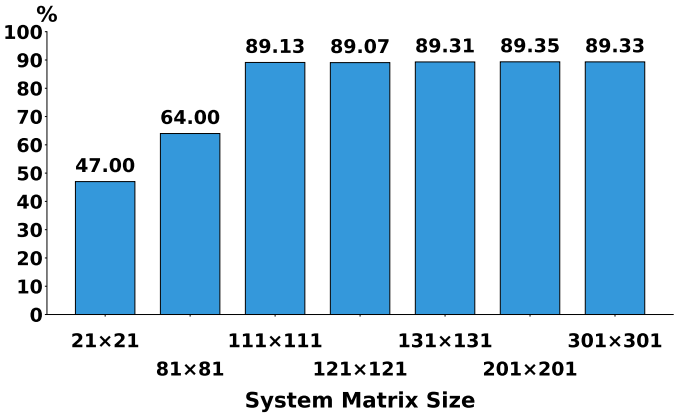


Fig. 2: Percentage of time spent on NNLS in MPI reconstructions across various system matrix sizes with MATLAB.

The core computational task is solving the NNLS problem in Equation (1), yielding particle concentration distribution $\mathbf{x}^*$. As shown in Fig. 2, the NNLS solver dominates overall runtime, accounting for over 89% of reconstruction time across all matrix sizes. For smaller matrices (21×21), preprocessing overhead is visible, but NNLS becomes increasingly dominant as problem size grows. This burden stems from iterative algorithms, intensive dense matrix operations, and element-wise non-negativity enforcement.

To achieve real-time MPI—especially for 3D or multi-coil settings—accelerating NNLS is essential. Sub-second reconstruction would enable intraoperative guidance applications currently infeasible. Fortunately, NNLS is highly data-parallel, well-suited for parallel CPU and GPU architectures. Dense matrix-vector multiplications decompose across output elements, and constraint projections are element-wise operations with no data dependencies. Our work explores optimized parallel NNLS solvers, including gradient projection and active-set methods, on CPUs and GPUs.

TABLE I: Parallel NNLS implementations: algorithm, hardware, parallel strategy, and key features.

Algorithm	Hardware Target	API	Parallel Strategy	Implementation Features
Active-Set	Multi-core CPU	OpenMP	Threaded active set and residual updates	Parallel constraint updates
Active-Set	GPU	CUDA	Kernels for set update; cuSOLVER	Fully on-GPU constraint management
Gradient Projection	Multi-core CPU	OpenMP	Threaded gradient and projection	Parallel gradient and projection
Gradient Projection	GPU	CUDA	Kernels for gradient and projection	All steps as GPU kernels

Algorithm 1 Accelerated Active-Set NNLS with OpenMP

Input: $\mathbf{A} \in \mathbb{R}^{m\times n}, \mathbf{b} \in \mathbb{R}^m$, tolerance ϵ
Initialize $\mathbf{x} \leftarrow \mathbf{0}$, active set $\mathcal{P} \leftarrow \emptyset$, passive set $\mathcal{R} \leftarrow \{1, \ldots, n\}$
while not converged **do**
  Compute gradient: $\mathbf{w} \leftarrow \mathbf{A}^T(\mathbf{A}\mathbf{x} - \mathbf{b})$ {Parallel}
  if $\max_{j\in\mathcal{R}} w_j \leq \epsilon$ **then**
    break {Optimality condition satisfied}
  end if
  parallel for $j \in \mathcal{R}$ **do**
  if $w_j > \epsilon$ **then**
    Move j from $\mathcal{R}$ to $\mathcal{P}$ {Update active set}
  end if
  end parallel for
  Extract submatrix $\mathbf{A}_\mathcal{P}$ (columns in $\mathcal{P}$)
  Solve $\mathbf{s}_\mathcal{P} \leftarrow \arg\min \|\mathbf{A}_\mathcal{P}\mathbf{s} - \mathbf{b}\|_2^2$ {QR via LAPACK}
  while $\min_{j\in\mathcal{P}} s_j \leq 0$ **do**
    Find $j^* \leftarrow \arg\min_{j\in\mathcal{P}} s_j/(s_j - x_j)$
    $\alpha \leftarrow s_{j^*}/(s_{j^*} - x_{j^*})$
    $\mathbf{x}_\mathcal{P} \leftarrow \mathbf{x}_\mathcal{P} + \alpha(\mathbf{s}_\mathcal{P} - \mathbf{x}_\mathcal{P})$
    Move all j with $x_j \leq 0$ from $\mathcal{P}$ to $\mathcal{R}$
    Re-solve with updated $\mathcal{P}$
  end while
  $\mathbf{x}_\mathcal{P} \leftarrow \mathbf{s}_\mathcal{P}$
end while
Output: $\mathbf{x}$

Algorithm 2 Accelerated Active-Set NNLS with CUDA

Input: $\mathbf{A} \in \mathbb{R}^{m\times n}, \mathbf{b} \in \mathbb{R}^m$, tolerance ϵ
Initialize $\mathbf{x} \leftarrow \mathbf{0}$ on GPU, active/passive sets on GPU
while not converged **do**
  Kernel launch: Compute $\mathbf{w} \leftarrow \mathbf{A}^T(\mathbf{A}\mathbf{x} - \mathbf{b})$
  Kernel launch: Find $w_{\max} \leftarrow \max_j w_j$ via parallel reduction
  if $w_{\max} \leq \epsilon$ **then**
    break
  end if
  Kernel launch: Update active/passive sets based on gradient
  Compact active indices using thrust::copy_if
  Extract active columns $\mathbf{A}_\mathcal{P}$ via gather operation
  Solve reduced system using cusolverDnDgels (QR-based)
  while minimum element in $\mathbf{s}_\mathcal{P} \leq 0$ **do**
    Kernel launch: Find minimum ratio for line search
    Kernel launch: Update $\mathbf{x}_\mathcal{P}$ with step size
    Kernel launch: Identify violated constraints
    Update sets and re-solve
  end while
  Kernel launch: Copy $\mathbf{s}_\mathcal{P}$ to $\mathbf{x}_\mathcal{P}$
end while
Output: $\mathbf{x}$ (copy to host if needed)

B. Parallel NNLS Implementations: APIs and Algorithms

To accelerate NNLS in MPI reconstruction, we implement both active-set and gradient projection algorithms on multi-core CPUs using OpenMP and GPUs using CUDA. Active-set methods converge in fewer iterations but involve complex QR factorizations of reduced subproblems. Gradient projection methods feature simpler iterations with element-wise operations, highly parallelizable but potentially requiring more iterations. Table I summarizes the four parallel solvers and their parallelization strategies. All implementations are carefully matched to hardware characteristics, optimizing memory layout for coalesced GPU access and CPU cache efficiency to ensure scalable and efficient NNLS acceleration.

1) Active-Set with OpenMP (on Multi-core CPU): The active-set solver parallelizes gradient computation, constraint membership updates, and projection steps (see Algorithm 1). Thread-level parallelism exploits OpenMP directives with dynamic scheduling to balance workload as the active set $\mathcal{P}$ varies across iterations. LAPACK routines handle QR factorizations of reduced systems.

2) Active-Set with CUDA (on GPU): Active set updates and projection execute in CUDA kernels, while Thrust library functions compact active indices and extract columns via gather operations (see Algorithm 2). The cuSOLVER library (cusolverDnDgels) solves reduced systems entirely on-device using QR-based methods, ensuring numerical stability while maintaining high throughput.

3) Gradient Projection with OpenMP (on Multi-core CPU): Gradient computation $\mathbf{g} = \mathbf{A}^T(\mathbf{A}\mathbf{x} - \mathbf{b})$ and projection to the non-negative orthant are parallelized across all variables using OpenMP (see Algorithm 3). Parallel BLAS routines accelerate matrix-vector products. The gradient computation dominates per-iteration cost and benefits significantly from multi-threading.

4) Gradient Projection with CUDA (on GPU): All vector operations execute as GPU kernels: cuBLAS gemv handles matrix-vector products, custom kernels perform element-wise projection, and parallel reductions compute norms (see Algorithm 4). Kernel fusion combines operations where possible to minimize memory traffic and maximize throughput.

Algorithm 3 Accelerated Gradient Projection with OpenMP

Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m$, max iterations K, tolerance ϵ
Initialize $\mathbf{x} \leftarrow \mathbf{0}$, α_0 (initial step size)
for $k = 1$ to K **do**
  Compute residual: $\mathbf{r} \leftarrow \mathbf{A}\mathbf{x} - \mathbf{b}$ {Parallel BLAS}
  parallel for $j = 1$ to n **do**
  $g_j \leftarrow (\mathbf{A}^T\mathbf{r})_j$ {Gradient component}
  end parallel for
  Compute $\|\mathbf{g}\|_2^2$ via parallel reduction
  if $\|\mathbf{g}\|_2 \leq \epsilon$ **then**
    break {Convergence}
  end if
  Backtracking line search:
    Set $\alpha \leftarrow \alpha_0$
  repeat
      parallel for $j = 1$ to n **do**
        $x_j^{\text{new}} \leftarrow \max(0, x_j - \alpha g_j)$ {Projection}
      end parallel for
      Evaluate objective $f(\mathbf{x}^{\text{new}}) = \|\mathbf{A}\mathbf{x}^{\text{new}} - \mathbf{b}\|_2^2$
      $\alpha \leftarrow \beta\alpha$ (with $\beta = 0.5$)
  until sufficient decrease condition satisfied
  $\mathbf{x} \leftarrow \mathbf{x}^{\text{new}}$
end for
Output: $\mathbf{x}$

Algorithm 4 Accelerated Gradient Projection with CUDA

Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m$, max iterations K, tolerance ϵ
Allocate and initialize $\mathbf{x} \leftarrow \mathbf{0}$ on GPU
for $k = 1$ to K **do**
  Kernel launch: Compute $\mathbf{r} \leftarrow \mathbf{A}\mathbf{x} - \mathbf{b}$ (cuBLAS gemv)
  Kernel launch: Compute $\mathbf{g} \leftarrow \mathbf{A}^T\mathbf{r}$ (cuBLAS gemv)
  Kernel launch: Parallel reduction for $\|\mathbf{g}\|_2^2$
  if $\|\mathbf{g}\|_2 \leq \epsilon$ **then**
    break
  end if
  Backtracking line search on GPU:
    Set $\alpha \leftarrow \alpha_0$
  repeat
      Kernel launch: $x_j^{\text{new}} \leftarrow \max(0, x_j - \alpha g_j)$ for all j
      Kernel launch: Compute $\mathbf{r}^{\text{new}} \leftarrow \mathbf{A}\mathbf{x}^{\text{new}} - \mathbf{b}$
      Kernel launch: Reduction for $\|\mathbf{r}^{\text{new}}\|_2^2$
      $\alpha \leftarrow \beta\alpha$
  until sufficient decrease or max backtrack iterations
  Kernel launch: $\mathbf{x} \leftarrow \mathbf{x}^{\text{new}}$
end for
Copy $\mathbf{x}$ to host memory
Output: $\mathbf{x}$

C. Unified Memory Support for GPU

MPI reconstruction problems, especially at high resolution or with multi-coil data, often generate system matrices and measurement vectors that exceed the physical memory available with GPUs. For instance, a 301×301 reconstruction with typical frequency encoding requires over 20 GB of memory in double precision. Unified memory enables seamless data management across CPU and GPU memory spaces, allowing the GPU to work on problem sizes over the capacity of VRAM. We evaluated all major CUDA unified memory options, including:

- **Naive unified memory**: `cudaMallocManaged` allocates both main and device memories to the GPU, relying on the CUDA driver's page fault mechanism for on-demand paging. It may incur significant overhead as pages migrate individually upon first access, causing GPU stalls.
- **Memory advice**: `cudaMemAdvise` is used to provide hints to the CUDA runtime about memory usage patterns, enabling optimizations such as preferred location, read-mostly access, or establishing direct mappings. In our implementation, we set `cudaMemAdviseSetPreferredLocation` to the GPU so that the device memory is favored, minimizing page migrations and reducing fault handling overhead.
- **Prefetch**: `cudaMemPrefetchAsync` proactively migrates entire memory regions to VRAM before kernel launch, overlapping transfer with computation on different streams. It eliminates page faults but requires explicit management of data movement timing and synchronization.
- **Combined advice and prefetch**: This applies both access advice and prefetching to optimize data locality and reduce runtime page migrations.

These unified memory strategies allow our GPU implementations to scale to data sets much larger than the onboard VRAM, facilitating truly large-scale MPI reconstruction and simplifying application development. While unified memory enables GPU acceleration beyond VRAM capacity, it incurs frequent page migrations between host and device, reducing performance compared to VRAM-only execution.

Furthermore, recent work has demonstrated that memory bandwidth is a decisive factor for the performance of data-parallel workloads [22], [23]. In the context of unified memory, host memory generally offers much lower bandwidth than VRAM (GDDR or HBM), and data transfers over PCIe suffer from more limited bandwidth. Therefore, unified memory may not be ideal for performance-critical cases where memory bandwidth is essential.

D. Floating-Point Precision: FP32 vs. FP64

Floating-point precision has major impacts on both accuracy and computational performance of the NNLS solvers. Lowering the precision helps halve the memory usage for all dense matrices and vectors, which not only improves bandwidth utilization but also enables larger problem sizes to fit into limited memory space. For example, a system matrix that

barely fits in 20 GB of VRAM in FP64 can accommodate problems nearly twice as large in FP32.

Furthermore, most modern GPUs offer significantly higher throughput for single precision (FP32) arithmetic compared to double precision (FP64), with FP64 often running at a fraction of FP32 performance. On our RTX 4000 Ada GPU, the theoretical peak FP32 throughput is 26.73 TFLOPS compared to only 417.6 GFLOPS for FP64—a ratio of approximately 64:1. This dramatic difference means that FP32 enables much faster computation on the GPU and is especially advantageous when absolute accuracy can be relaxed or memory constraints are tight. The trade-off is potential accumulation of rounding errors during iterative solves, which we quantify in our experimental evaluation. Consequently, our implementations are evaluated with both FP64 and FP32 to characterize the performance-accuracy trade-off space.

TABLE II: Specifications of the Hardware Platforms.

Specification	**CPU: i9-14900T**	**GPU: RTX 4000 Ada**
Cores/Threads	24 / 32	6,144 CUDA cores
Base/Boost Freq.	1.1 / 5.1 GHz	1.5 / 2.175 GHz
Peak FP64 (DP)	908.8 GFLOPS	417.6 GFLOPS
Peak FP32 (SP)	1.82 TFLOPS	26.73 TFLOPS
Memory	192 GB DDR5	20 GB GDDR6
Memory BW	76.8 GB/s	360 GB/s
PCIe BW	63.015 GB/s	63.015 GB/s
TDP	35 W	130 W

IV. Experimental Evaluations

A. Experimental Setup

Experiments were performed on a workstation with Intel Core i9-14900T CPU (24 cores/32 threads; turbo boost is turned off for stable core frequencies) and NVIDIA RTX 4000 Ada Generation GPU (6,144 CUDA cores, 20 GB GDDR6). The system has 192 GB DDR5-4800 memory and runs Ubuntu 22.04.5 LTS with NVIDIA GPU driver 570.124.06, CUDA Toolkit 12.8, GCC 11.4 (OpenMP 4.5) and MATLAB R2024b. All software was compiled for maximum optimization.

Table II summarizes the CPU and GPU specifications, including peak FP32 and FP64 throughputs. The Intel i9-14900T has 35 W TDP with 106 W peak power during boost intervals. MATLAB's `lsqnonneg` was evaluated in both single- and multi-threaded modes (via Parallel Computing Toolbox), with the latter serving as our baseline as it represents the practical configuration used with multi-core systems. We selected MATLAB as it represents the most commonly used NNLS solver in MPI research, providing a realistic reference for typical workflows. While specialized solvers like TNT-NN [16] exist, we focus on demonstrating how fundamental algorithmic approaches (active-set and gradient projection) accelerate across parallel architectures, providing insights applicable to various NNLS implementations.

System matrices were generated from PSF measurements, with test sizes ranging from 21×21 up to 301×301 per plane, and realistic 3D multi-coil datasets covering 25–55 mm depth in 2 mm steps. Tikhonov regularization ($\lambda = 0.001$ to 0.1) was used to stabilize reconstructions.

On our experimental platform, the largest system matrix that can be stored entirely in VRAM using double precision is 131×131. This limit is set by the 20 GB memory capacity of the RTX 4000 Ada GPU and the quadratic growth in memory usage with matrix size. Larger system matrices require unified memory support for this GPU to access the main memory.

B. Evaluation Results

We systematically evaluated all four parallel NNLS implementations across system matrix sizes, hardware targets, algorithms, and numerical precisions. Speedup results, normalized to the MATLAB multi-threaded baseline, are summarized in Fig. 3. Baseline execution times with MATLAB are: *0.0366s* for 21×21, *13.86s* for 81×81, *73.26s* for 111×111, *118.29s* for 121×121, *125.57s* for 131×131, *533s* for 201×201, and *1727s* for 301×301. Note that MATLAB's `lsqnonneg` only supports FP64.

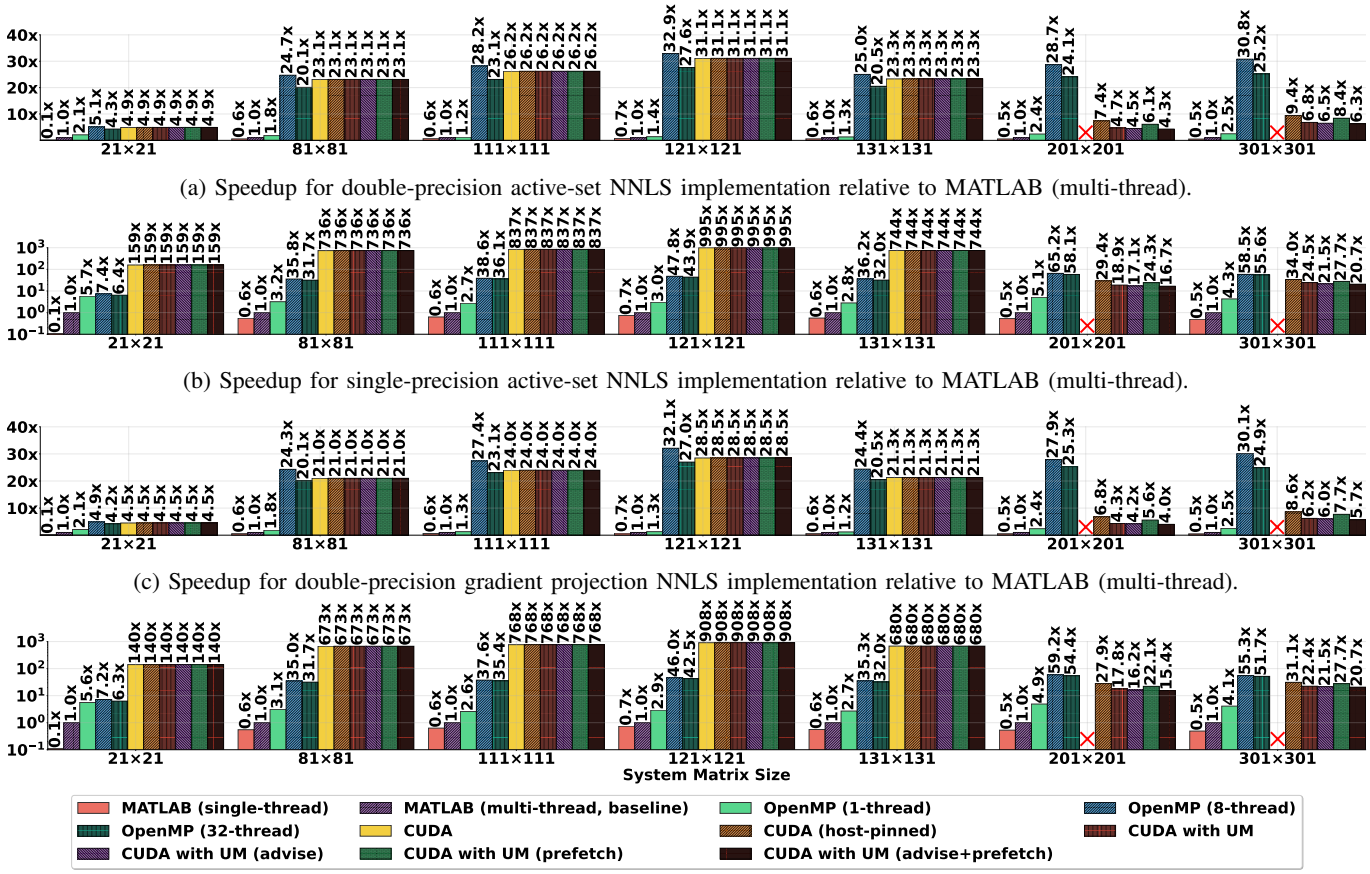
1) Double-Precision Performance: Figures 3a and 3c present FP64 results. For small matrices (21×21), speedups are modest across all platforms as fixed overheads (kernel launch latency, thread creation) dominate execution time. As matrix size increases, both OpenMP and CUDA yield substantial acceleration. With a 131×131 system matrix, CUDA GPU achieves up to $23\times$ speedup for active-set and $21\times$ for gradient projection. OpenMP (8 threads) achieves 24–25$\times$ speedup, confirming effective multi-core scaling.

Interestingly, performance gains are slightly higher on multi-core CPU versus GPU for double precision, as the RTX 4000 Ada GPU's FP64 performance is limited (417.6 GFLOPS versus 908.8 GFLOPS on CPU). This demonstrates that for FP64-critical applications, high-end CPUs with many cores can compete with mid-range GPUs. Gradient projection shows slightly lower speedups than active-set for both platforms, attributable to simpler per-iteration operations that amortize parallelization overhead less effectively.

2) Single-Precision Performance: Figures 3b and 3d show FP32 results. Here, GPU performance advantage is tremendous: CUDA implementations reach speedups exceeding $100\times$ (up to $995\times$) on 131×131 matrices, due to reduced memory usage and much higher single-precision throughput. This dramatic acceleration stems from the GPU's 64:1 FP32 to FP64 throughput ratio, combined with doubled memory bandwidth utilization from halved data size. OpenMP achieves up to 35–36$\times$ speedup with 8 threads, demonstrating that multi-core CPUs also benefit from reduced precision, though to a lesser extent. Overall, GPU consistently delivers highest performance under FP32 for matrix sizes up to 131×131.

The absolute reconstruction times with GPU FP32 are clinically significant: for 131×131 matrices, reconstruction completes in approximately 126 milliseconds, well within sub-second requirements for real-time interventional imaging. Even for 81×81 matrices, GPU FP32 achieves reconstruction under 20 milliseconds, enabling potential frame rates exceeding 50 Hz for real-time tracking.

Fig. 4 shows Euclidean distance between FP32 and FP64 reconstructions, increasing from 2.25×10^{-3} for 21×21 to



(a) Speedup for double-precision active-set NNLS implementation relative to MATLAB (multi-thread).

(b) Speedup for single-precision active-set NNLS implementation relative to MATLAB (multi-thread).

(c) Speedup for double-precision gradient projection NNLS implementation relative to MATLAB (multi-thread).

(d) Speedup for single-precision gradient projection NNLS implementation relative to MATLAB (multi-thread).

Fig. 3: Measured speedup of all parallel NNLS implementations over the MATLAB multi-threaded baseline, for both active-set and gradient projection algorithms, in double and single precision. Results are shown as a function of system matrix size. GPU (CUDA), multi-core CPU (OpenMP), and unified memory configurations are included.

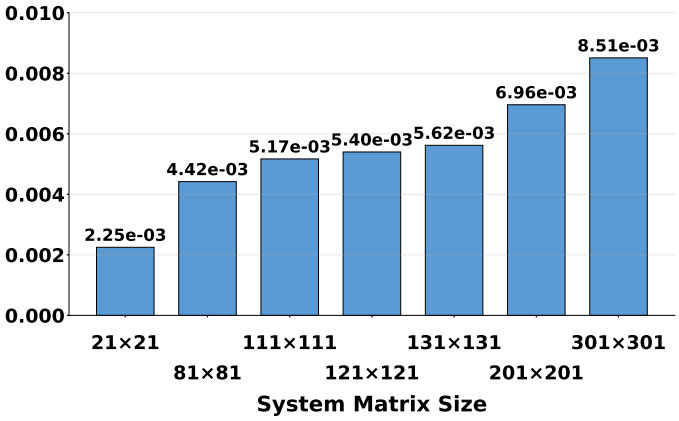


Fig. 4: Euclidean distance between FP32 and FP64 reconstructions across various system matrix sizes.

8.51×10^{-3} for 301×301 matrices, demonstrating gradual but controlled growth. Despite this increase, error remains well below clinical thresholds (typically < 0.1 for normalized reconstructions), particularly important as GPU vendors increasingly prioritize lower-precision arithmetic. The sublinear error growth suggests FP32 rounding error accumulation is well-controlled by NNLS algorithms.

3) Unified Memory Scalability: We evaluated different unified memory strategies in CUDA. All tested options—naive managed memory, memory advice, prefetching, and their combination—enable reconstruction of datasets larger than GPU physical memory, supporting matrix sizes up to 301×301. Explicit prefetching and memory advice provide best performance, though differences are generally small for well-optimized kernels. However, unified memory approaches deliver worse performance than direct host memory access (host-pinned cases). While enabling reconstruction beyond VRAM capacity, unified memory incurs substantial performance penalties from page migrations between host and device.

Moreover, host-pinned cases (with PCIe data transfers) suffer from limited PCIe bandwidth (63 GB/s), severely constraining GPU performance (up to $34\times$ with FP32) compared to multi-core CPU (up to $59\times$ with FP32). This crossover point—where CPU becomes faster than GPU with unified memory—highlights memory bandwidth's critical importance for data-intensive reconstruction. For datasets exceed-

ing VRAM, practitioners should consider either upgrading to GPUs with larger memory (e.g., 48 GB professional cards) or accepting unified memory's performance tradeoff for simplified programming.

4) Further Discussions: The experimental results highlight several insights for MPI reconstruction. First, MATLAB's poor performance stems from interpreted language execution overhead and suboptimal memory access patterns in `lsqnonneg`.

Second, OpenMP with 8 threads (up to $59\times$ with FP32) outperforms 32 threads (up to $56\times$ with FP32) on Intel Core i9-14900T due to its heterogeneous architecture (8 P-cores+16 E-cores). Performance disparity between faster P-cores and slower E-cores creates synchronization bottlenecks at barrier points. Thread affinity policies preferentially using P-cores could further improve performance.

Third, unified memory enables efficient reconstruction for problems exceeding VRAM, making GPU acceleration accessible for high-resolution datasets without manual memory management, though overhead remains from page migrations.

Fourth, while our evaluation covers matrix sizes up to 301×301, this range encompasses clinically relevant problem sizes for current MPI scanner geometries. Larger matrices would require proportionally larger fields-of-view or higher spatial sampling beyond present hardware capabilities. The consistent scaling trends observed suggest our acceleration strategies generalize to future higher-resolution systems.

Finally, results suggest future opportunities in hybrid precision schemes, algorithmic optimizations exploiting tensor cores, and multi-GPU or distributed implementations for ultra-high-resolution reconstructions exceeding single-node memory capacity. Algorithm-hardware co-design with intelligent memory management is crucial for advancing practical, scalable MPI reconstructions.

V. Conclusions

This work presents a comprehensive comparative study of hardware acceleration strategies for NNLS reconstruction in MPI. Through systematic evaluation of OpenMP multi-core CPU optimization, CUDA GPU acceleration, unified memory management, and reduced-precision implementations, we provide practical guidance for selecting optimal strategies based on available hardware resources.

Our key contributions include: (1) systematic comparison of active-set versus gradient projection methods across platforms; (2) unified memory strategies enabling reconstruction beyond VRAM limits; (3) FP32 versus FP64 trade-off characterization demonstrating clinical accuracy with dramatic speedups. The dramatic speedups—over $900\times$ for GPU FP32 and $59\times$ for multi-core CPU—enable sub-100 millisecond reconstruction times, transforming MPI from a pre-operative tool to an intra-operative guidance system for applications such as sentinel lymph node biopsy.

Acknowledgments

First and foremost, we sincerely thank the anonymous reviewers for their valuable comments. In addition, the authors would like to acknowledge that a generative AI tool (Claude) was used to only assist with grammar and spelling refinement during the preparation of this manuscript. This work was supported, in part, by JSPS KAKENHI with Grant JP22K11958 from Japan.

References

[1] B. Gleich *et al.*, "Tomographic imaging using the nonlinear response of magnetic particles," *Nature*, vol. 435, pp. 1214–1217, 2005.

[2] T. Knopp *et al.*, *Magnetic particle imaging: an introduction to imaging principles and scanner instrumentation*. Springer, 2012.

[3] J. Weizenecker *et al.*, "Three-dimensional real-time in vivo magnetic particle imaging," *Physics in Medicine & Biology*, vol. 54, no. 5, pp. L1–L10, 2009.

[4] S. Bai *et al.*, "Development of a human-size magnetic particle imaging device for sentinel lymph node biopsy of breast cancer," *Frontiers in Bioengineering and Biotechnology*, vol. 12, no. 1327521, pp. 1–7, 2024.

[5] L. Zhang *et al.*, "Current reconstruction approaches of magnetic particle imaging: a review," *Journal of Magnetism and Magnetic Materials*, vol. 594, no. 171894, pp. 1–18, 2024.

[6] T. F. Sattel *et al.*, "Single-sided device for magnetic particle imaging," *Journal of Physics D: Applied Physics*, vol. 42, no. 2, pp. 1–5, 2008.

[7] F. Xu *et al.*, "Accelerating popular tomographic reconstruction algorithms on commodity PC graphics hardware," *IEEE Transactions on Nuclear Science*, vol. 52, no. 3, pp. 654–663, 2005.

[8] S. S. Stone *et al.*, "Accelerating advanced MRI reconstructions on GPUs," *Journal of Parallel and Distributed Computing*, vol. 68, no. 10, pp. 1307–1318, 2008.

[9] G. Pratx *et al.*, "GPU computing in medical physics: a review," *Medical Physics*, vol. 38, no. 5, pp. 2685–2697, 2011.

[10] J. Dieckhoff, D. Eberbeck, M. Schilling, and F. Ludwig, "Magnetic-field dependence of Brownian and Néel relaxation times," *Journal of Applied Physics*, vol. 119, no. 4, p. 043903, 2016.

[11] C. L. Lawson *et al.*, *Solving Least Squares Problems*. SIAM, 1995.

[12] V. Franc *et al.*, "Sequential coordinate-wise algorithm for the non-negative least squares problem," in *Computer Analysis of Images and Patterns*, ser. Lecture Notes in Computer Science, vol. 3691, 2005, pp. 407–414.

[13] T. Vu *et al.*, "On asymptotic linear convergence of projected gradient descent for constrained least squares," *IEEE Transactions on Signal Processing*, vol. 70, pp. 4061—-4076, 2022.

[14] A. Beck *et al.*, "A fast iterative shrinkage-thresholding algorithm for linear inverse problems," *SIAM Journal on Imaging Sciences*, vol. 2, no. 1, pp. 183–202, 2009.

[15] S. Boyd *et al.*, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.

[16] J. M. Myre *et al.*, "TNT-NN: A fast active set method for solving large non-negative least squares problems," *Procedia Computer Science*, vol. 108, pp. 755–764, 2017.

[17] Z. Tian *et al.*, "Low dose CT reconstruction via edge-preserving total variation regularization," *Physics in Medicine and Biology*, vol. 56, no. 18, pp. 5949–5967, 2011.

[18] Y. Luo *et al.*, "Efficient parallel non-negative least squares on multi-core architectures," *SIAM Journal on Scientific Computing*, vol. 33, no. 5, pp. 2848–2863, 2011.

[19] E. Battenberg *et al.*, "Accelerating non-negative matrix factorization for audio source separation on multi-core and many-core architectures," in *Proc. of ISMIR'09*, 2009, pp. 501—506.

[20] S. Sánchez *et al.*, "GPU implementation of fully constrained linear spectral unmixing for remotely sensed hyperspectral data exploitation," in *Proc. Vol. 7810: Satellite Data Compression, Communications, and Processing VI*, vol. 7810G, 2010, pp. 1–11.

[21] S. Kestur *et al.*, "BLAS comparison on FPGA, CPU and GPU," in *Proc. of ISVLSI'10*, 2010, pp. 288–293.

[22] J. Zheng *et al.*, "Exploiting data parallelism in graph-based simultaneous localization and mapping: A case study with GPU accelerations," in *Proc. of HPC-Asia'23*, 2023, pp. 126–139.

[23] J. Wu *et al.*, "Accelerating graph-based slam through data parallelism and mixed precision on FPGAs," in *Proc. of MCSoC'23*, 2023, pp. 284––292.